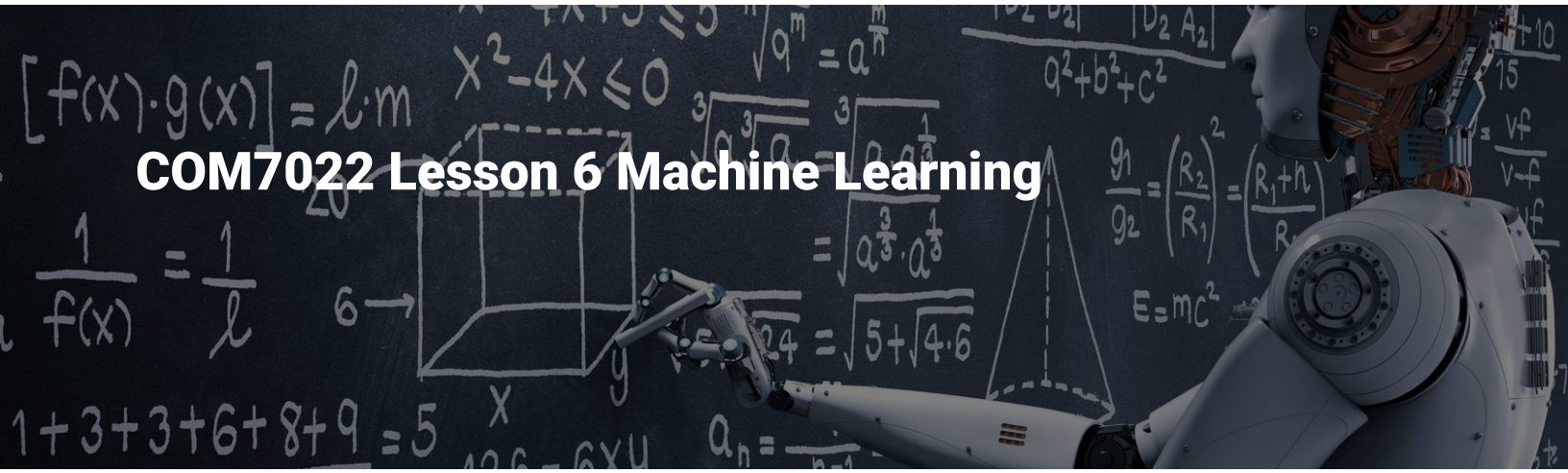


COM7022 Lesson 6 Machine Learning

Machine Learning (COM7022)

Lesson 6: Naive Bayesian

Cover image by Phonlamai Photo from Shutterstock.com

- **Lesson introduction**
- **Before you get started**
- **Topic 1: Naïve Bayesian**
- **Topic 2: How to use Bayes Theorem for Classification**
- **Topic 3: FULL example of a Naïve Bayesian classifier using imported data**
- **Topic 4: Bayesian Belief Network**
- **Knowledge check**

Lesson summary

Reading List

Lesson introduction

Welcome to Lesson 6

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

Welcome to Lesson 6 on Naïve Bayesian.

Naïve Bayesian is heavily used within probability data science studies, which is why it has been included within this module. Initially, we will be introducing the topic from a graphical and mathematical perspective so that we can get a firm grasp on the basic principles behind the classifier. Once the initial theory has been demonstrated, we shall dive a little deeper and show, through examples, how to use the classifier effectively and detail the steps required to build a successful model, based upon independent contributions of probability: all of which will be building up to the full, in-depth example given within the middle section of the lesson. Finally, we demonstrated what a Bayesian Belief Network is.



Computing

At Arden University

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

By the end of this lesson you will:

- Evaluate what Naïve Bayesian is and how it is used within a probability classifier environment
- Formulate the ideal steps to be taken to successfully apply the classifier
- Have a full, rounded knowledge of how to apply a Naïve Bayesian classifier
- Determine what a Bayesian Belief Network is and how to evaluate one for an associated probability study

Lesson 6 aligns to Learning Objective 1 and 2:

- Demonstrate a practical understanding of the characteristics of various Machine Learning methods and their applications in data analysis.

- Select and apply appropriate machine learning techniques for a given scenario



We have worked across teams at Arden University to try and create an accessible and inclusive experience within the learning content. We are continuously assessing the accessibility of our lessons to make improvements and enhancements based on testing and feedback.

This lesson may include screenshots of code and software used. Please be aware that these may not always have a text alternative, and that other features such as size and colour may not be accessible.

If you experience any issues with accessing the learning content throughout this module, there are several support services in place to help you.

- Arden University has a dedicated Inclusion Services Team who can help you with getting adjustments and equipment.
- The Library Portal also has information on the accessibility tools that can help you access resources: Accessibility Tools – Library Portal
- If you are struggling with accessing a specific part of the content e.g. a video resource that is missing captions, please speak to your Lecturer who may be able to advise on an alternative resource in the interim to ensure you are able to get the learning you need to progress with the lesson.

Continue to the Before you get started section

Before you get started



Image from Articulate Rise 360 Content Library.

Did you know?

A study of over 58,000 (mostly American) Facebook users showed that people who “liked” curly fries tended to have higher IQs than those who didn’t (University of Cambridge, 2013).



Image from Articulate Rise 360 Content Library.

Continue to Topic 1

Topic 1: Naïve Bayesian



Image from Articulate Rise 360 Content Library.

Naive Bayes classifiers are a collection of classification algorithms based on Bayes' Theorem. It is not a single algorithm, but similar to Random Forest, a family of algorithms all sharing a common principle, which is **every pair of features being classified is independent of each other**. Therefore, a typical investigation would be classifying the weather conditions and whether a person would be able to play tennis or not, given the conditions as fit ("Yes") or unfit ("No").



To be as statistically correct as possible, when an analysist wants to investigate, or learn about Naïve Bayesian, they are more often than not interested in learning about the Multinomial Naïve Bayesian Classifier, which is what we are going to demonstrate here. However, there are other applied Naïve Bayesian methods, one of which is the Gaussian Naïve Bayesian Classifier.

Multinomial Naïve Bayesian Classifiers are predominately, but not limited to, text anomaly detection and used continuously within emails sent by various parties. The classifier is written into the email servers or additional software to help determine whether the email sent is spam or not. It is this scenario which we shall be using as our example.



Let’s build the scenario: say that the classifier which is running on our emails has a bug in it and not working correctly. We want to build a classifier that will determine, as best as possible using historical data, new emails sent from our Friends and Family (F&F) from Spam (S) emails. To do this we will use key words found in both types of emails.

Therefore, we have a collection of historical emails, where green are F&F and red S, shown in Figure 6.01.

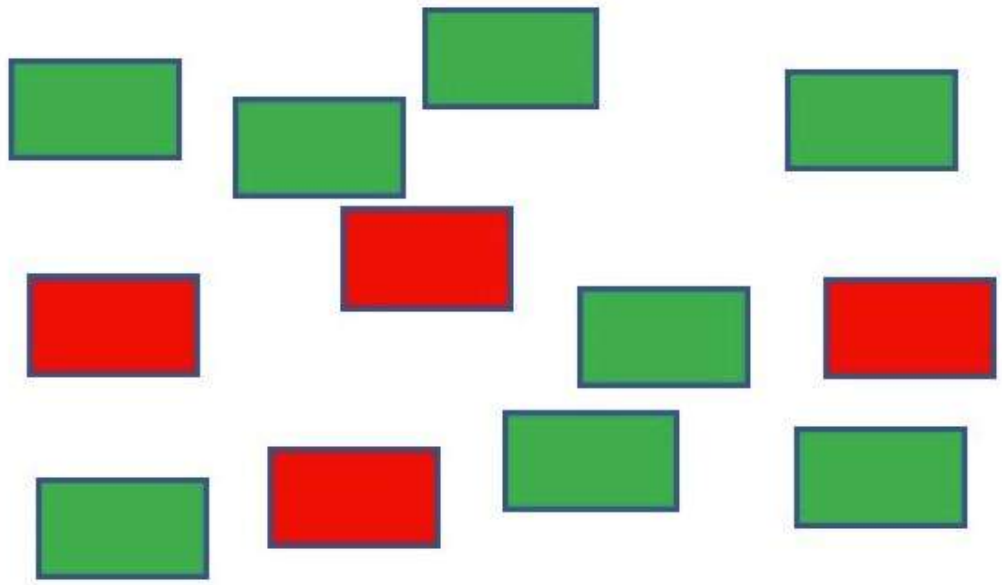
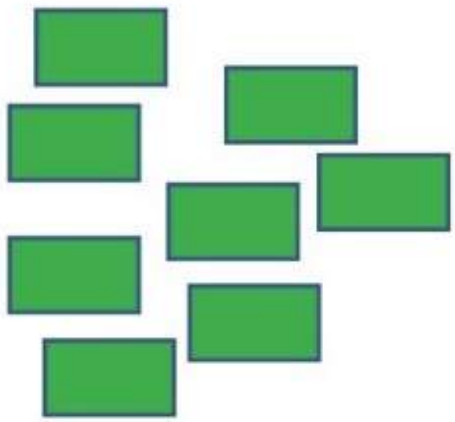
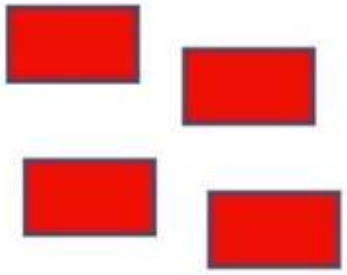


Figure 6.01. A collection of historical email from F&F and S by Arden University.

The first step is to separate out F&F and S emails. The second step is to pick out frequently used key words, from both sets of emails. The third step is to tally up the frequency of these key words, as shown in Figure 6.02.



	Dear	Friend	Lunch	Money
Freq	8	5	3	1
P	0.47	0.29	0.18	0.06



	Dear	Friend	Lunch	Money
Freq	2	1	0	4
P	0.29	0.14	0.00	0.57

Figure 6.02. The separation and frequency distribution of key words from each set of emails by Arden University.

Now that we have chosen our key word and tallied them up, we can use this information to determine each probability for every key word in both tables. To do this we simply say, what is the given the probability that the key word “Dear” will be seen in a F&F message. Mathematically this is written:

$$P(Dear \mid F\&F) = \frac{8}{17} = 0.47$$

We can now repeat this for all key words using the same mathematical method, which is where the P row comes from in the above tables. It is important to note that in the above we have used discrete values, not continuous values, such as weight or height.

Having now determined all the probabilities above on historical data, a new data point (email) is introduced and this new data point begins the email with “Dear Friend”. The idea now is to determine, using the previously calculated probabilities, whether the new email is F&F or S.

We begin to do this with an initial guess about the probability, that any message, irrelevant of the content, is F&F, which we shall denoted as p\left(F&F\right). The guess can be whatever probability you choose, but a sensible guess would be estimated from the training data. As an example, since 8 of the 12 emails are F&F and 4 S, our initial guess will be 0.67. This is shown mathematically below.

$$p(F\&F) = \frac{8}{8+4} = 0.67$$

Our initial probability guess now determined, we need to investigate the probabilities of each word in “Dear friend”. Therefore, we multiply the initial guess by the probabilities we calculated earlier, i.e. Dear Friend, that appears in a F&F email. In other words:

$$p(F\&F) * p(Dear|F\&F) * p(Friend|F\&F)$$

$$0.67 * 0.47 * 0.29 = \mathbf{0.09}$$

We can think of this as a score for Dear friend, if it is a F&F email. We now do exactly the same process for the probability of Dear Friend being S, as shown below.

$$p(S) = \frac{4}{4+8} = 0.33$$

$$p(S) * p(Dear|S) * p(Friend|S)$$

$$0.33 * 0.29 * 0.14 = \mathbf{0.01}$$

As before with F&F emails, we can think of this as a score for Dear friend, if it is S.

To conclude then and to make the classification, we take the two results of **0.09** and **0.01** and determine which has the highest probability. Of course, in this example it is **0.09**, therefore we state that the new email we received was indeed a F&F email.

To sum up, we took key words out of the historical emails, placed their frequency in a table and calculated each probability for each key word. A new data point (email) was then added to the data and using the pre-calculated probabilities determined the classification of the email. We will finish it here, but in order to fully determine the classification, we would repeat the process with all combinations of key words.

Continue

Topic 1 - Activity 1

Discussion on iLearn - Naïve Bayesian and the Bayesian Theorem



Further research Naïve Bayesian and the Bayesian Theorem from fundamental principle. Determine what type of statistical methods this particular probability theory is associated to.

Using the mathematical example above, change the initial numerical values to whatever you like and work the problem through. State in the groups discussion forum what values you used and what your final solution represents in plane text.

Continue

Topic 1 - Activity 2

Reading



- Chapter 1: Introduction (pp. 11-31) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido
- Chapter 2: Anaconda, Spyder and the Libraries NumPy, Matplotlib and SymPy (pp. 41-52) from Python for Scientific Computing and Artificial Intelligence by Lynch
- Chapter 1: Preliminaries (pp. 12-30) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney
- Chapter 1: Data, Stats and Stories – An Introduction (pp. 39-70) from Statistics and Data Visualisation with Python by Rogel-Salazar

Continue

TOPIC REVIEW

Well done, you have completed Topic 1.

The initial topic on Naïve Bayesian has been concerned with not only introducing the topic, but also its graphical and mathematical representation. It is imperative that we

understand this popular classifier from base principles, i.e., the mathematics. It is only through detailing this that we can understand what Naïve Bayesian does and the ideology behind such an algorithm. Hopefully, you now fully understand the basic principles behind the classifier which will allow us to develop further in the next topic, i.e., how you use the Naïve Bayesian classifier effectively.

See you there.

Continue to Topic 2

Topic 2: How to use Bayes Theorem for Classification



Image from Articulate Rise 360 Content Library.

Did you know?

By performing association rule mining, a 2020 study of students on an online university course in the USA found that students who posted regularly on the discussion forums were less likely to get a low exam result

than other students (Valdiviejas and Bosch, 2020).

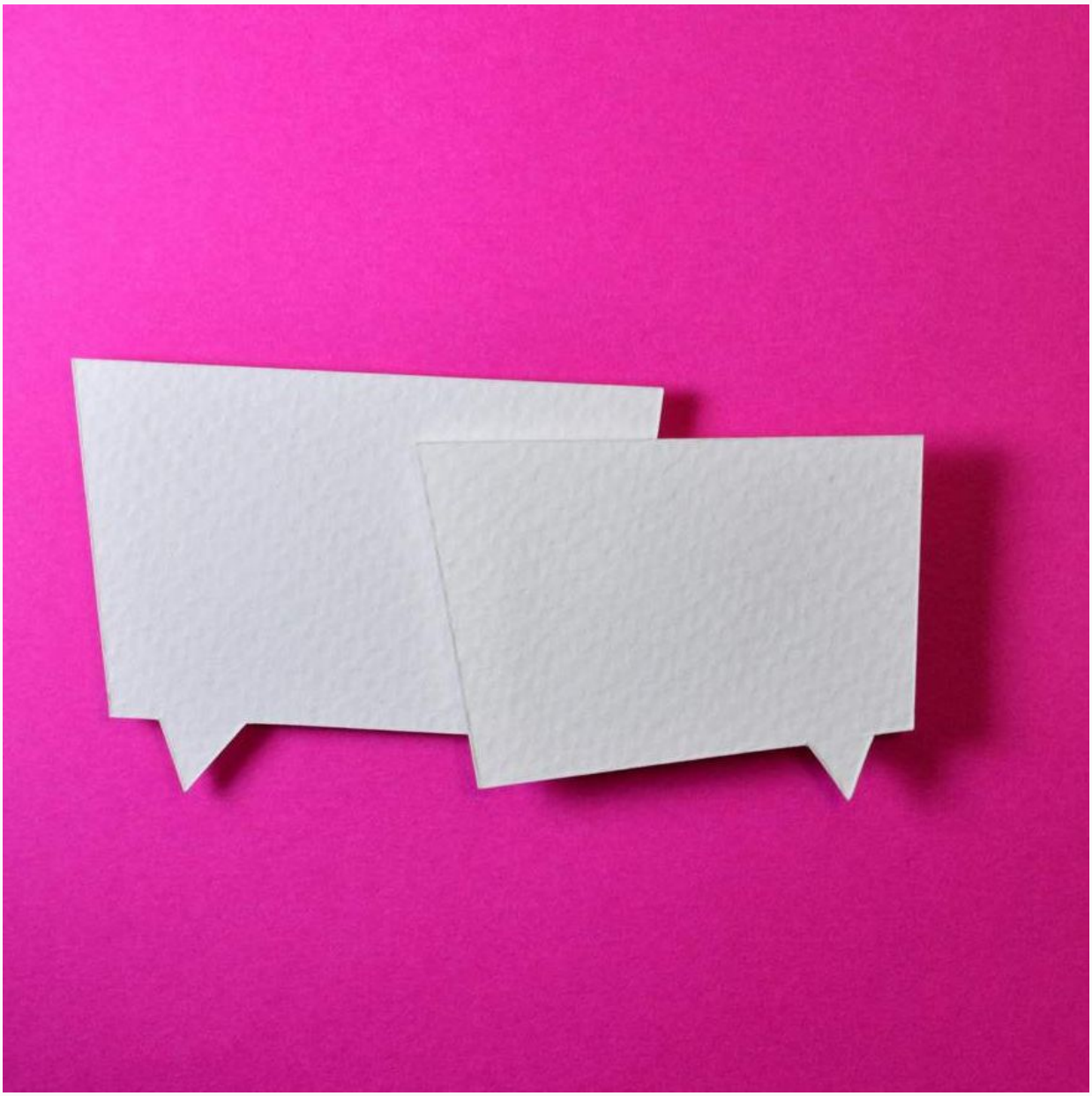


Image from Articulate Rise 360 Content Library.

Continue

The Naïve Bayesian Classifiers (NBC) are an ideal model to use when a data scientist is looking for a quick, fast, classification algorithm. Included within this topic, which is building upon topic one, we shall be demonstrating how the algorithm works and its many benefits, including a fully coded example.

The NBC, as we saw in Topic 1, separates the data into different classes according to the Bayes Theorem, always keeping in mind that all predictions are independent of each other. So, for example, we could consider a fruit to be an orange if it is round, orange and 6 cm in diameter. Each one of these features could depend on each other, but each one independently contributes to the probability that the fruit is an orange.



Image from Articulate Rise 360 Content Library.

Continue

Looking at this in an alternative manner, similar to a previous example, say we are looking at weather conditions for a rugby game. Every tuple will categorise the conditions as either go (Yes) or no-go (No) for the game going ahead. Therefore, the data set it split, or categorised into two parts, namely the response vector and feature matrix. The response vector, known as the class variable, is the prediction for every row in the matrix. Staying with the rugby example again, the response variable name would be “Play Rugby”.

The feature matrix includes all the vectors, i.e., rows, of the data set, which would include all the dependent variables such as temperature, windy, rainy and humidity.

The algorithm assumes that each feature will make an independent and equal contribution to the outcome, thus further assuming that no feature is dependent. In other words, the feature temperature is not related to humidity, or the feature windy is not related to rainy. Which is why all features contribute to the final outcome.

Continue





Image from Articulate Rise 360 Content Library.

Back to the Rugby. The Six Nations final is being played in Ireland and you have been asked to give the prediction of whether the game is to go ahead based on historical weather data for that particular weekday. Therefore, the targeted variable is going to be “Game On”. The steps below are how you would ideally approach the problem.

1. Transform the data set into frequency tables.
2. By determining the probabilities of the given feature, create a likelihood table.
3. To determine the posterior probability, use Bayes Theorem.

When implementing the classifier, there is no single choice. In fact, there are three main types of NBC’s in which to choose and these are detailed below.

Bernoulli Naïve Bayes

In this classifier the predictor is Boolean in nature. Therefore, the only answers available are going to be either true, or false, or yes / no. The only time that you would apply this classifier is when the Bernoulli distribution is multivariate. As the classifier deals with independent Boolean variables, the algorithm will result if a particular word is included, or not within any given document, which is why it is ideal for document classification studies.

Multi Nominal Bayes

This Bayes Classifier is used for document classification purposes. Say for example a data scientist was investigating, a large number of historical documents and need to determine which document belonged to which department. In other words, classifying whether each document belonged to say the HR department, or an Academic resource department, or a Finance department. The classifier achieves this by assessing the frequency of specific words contained within the documents, which in turn has the possibility of saving a considerable amount of time.

Gaussian Naïve Bayes

If the predictors are not discrete, yet have continuous values, the chances are that the sample used within the algorithm follow a Gaussian Distribution. In other words, the study assumes a normal distribution. More often than not, this is the main Naïve Bayesian Classifier that it used.

As a recap to Topic 1, the steps required to engage in such an investigation can be broken down onto multiple steps, thus guiding us through to successful completion.

1. Data pre-processing: efficient preparation of the data set for further study.
2. Fitting the Naïve Bayesian to the data set: split the data into training a testing set and fit the algorithm to the training set to obtain initial model accuracy.

3. Prediction of the test set: create a new predictor variable and use for predictions. If the test set gives predictions which are different to the test set, then the predictions are incorrect and alterations within the algorithm / data set are sort.

4. Create a confusion matrix: in this step we are testing the model's accuracy and the creation of a Confusion Matrix helps the data scientist to determine this.

5. Visualise results.

Let us now look at an example so that we can adopt theory to a practical environment. Here we will be using the **make_classification** function, which generates a random data set from the **scikit-learn.dataset** module. We will define five features, which are the target variables and four classes, **n-samples** are the number of observations in the data set, here we have stated 1000. The **n_informative** function is the number of informative features within the data set and in this example, we have defined these to be 2. We also need a **random_seed** function so that we can ensure that the data set is reproducible, all of which is shown in Figure 6.03 below.

```
from sklearn.datasets import make_classification

X, y = make_classification(
    n_features=5,
    n_classes=4,
    n_samples=1000,
    n_informative=2,
    random_state=1,
    n_clusters_per_class=1,
)
```

Figure 6.03. Importing the relevant library and creating the models environment in Google Colab by Arden University. (Accessible code).

Next we import matplotlib so that we can use the **scatter()** function to visualize the data: the scatter function takes three arguments, i.e. **X[:, 0]** and **X[:, 1]**, which are the first and second

column of the *X* array. The **marker** argument will specify the shape of each data point to be generated and here we have used circles. The resulting scatter plot which can be seen below in Figure 6.04 will have the first column of *X* along the x – axis and the second column of *X* along the *y* – axis. The colours are representative of the corresponding values in *y*.

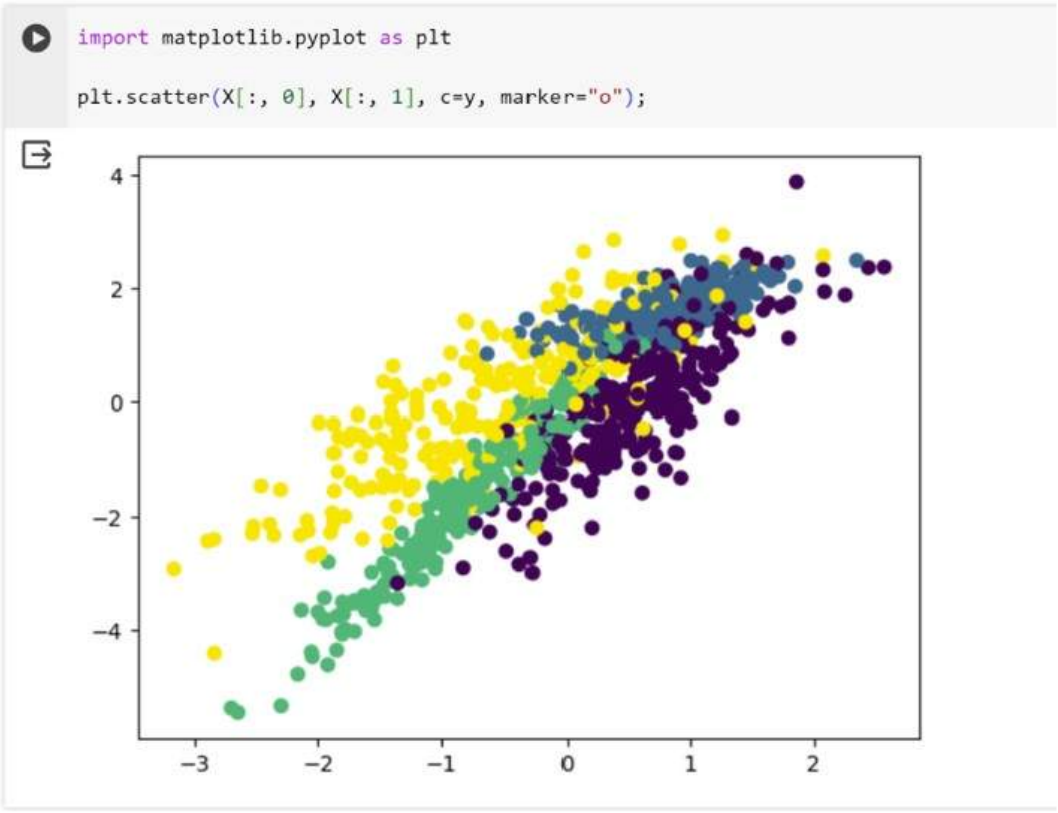


Figure 6.04. The resulting scatter plot of four classes and five features in Google Colab by Arden University. (Accessible code).

We now import the train / test function from the scikit – learn model selection, which splits the data set into training and testing data sets. In essence, the **test_train_split** creates four arguments, *X*, *y*, **test_size** and **random_state**, where *X* and *y* are the input features and target variables. The **test_size** is the defined percentage that we have assigned to the test data, her we have assigned 0.70, or in other words 70%. The **random_state** function is there so that we can run the random

number generator so that the same random split is applied each time the code is run, which can be seen in Figure 6.05 below.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.70, random_state=125
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.70, random_state=125
)
```

Figure 6.05. Splitting the data set into train and test in Google Colab by Arden University. (Accessible code).

The following code shown in Figure 6.06 below demonstrates that we have imported the Gaussian Naïve Classifier and a new instance of the **GaussianNB** class is created and assigned to the new variable **model**, which is then trained using the fit() method, which takes the training data, **X_train** and the corresponding targeted values of **y_train**.

```
from sklearn.naive_bayes import GaussianNB

# Build a Gaussian Classifier
model = GaussianNB()

# Model training
model.fit(X_train, y_train)

# Predict Output
predicted = model.predict([X_test[6]])

print("Actual", y_test[6])
print("Predicted", predicted[0])
```

```
Actual 0
Predicted 0
```

Figure 6.06. Running the GaussianNB algorithm and showing predicted and actual results being the same in Google Colab by Arden University. (Accessible code).

The next step we need to evaluate the model, the functions **accuracy_score**, **confusion_matrix**, **ConfusionMatrixDisplay** and **fi_score**, which are all imported from the **sklearn.metric** module aid the data scientist to do this. Which can be seen in Figure 6.07 below, including the results.

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score,
)

y_pred = model.predict(X_test)
accuray = accuracy_score(y_pred, y_test)
f1 = f1_score(y_pred, y_test, average="weighted")

print("Accuracy:", accuray)
print("F1 Score:", f1)
```

```
Accuracy: 0.7214285714285714
F1 Score: 0.7287524678048803
```

Figure 6.07. Initial results of the GaussianNB model, including accuracy result in Google Colab by Arden University. (Accessible code).

As we can see for Figure 6.07 above, the model has not performed to badly, considering that default hyperparameters we applied. The accuracy score tells us that the model predicted the correct results 72%. The F1 score of 73% is the measure of the model's accuracy, taking into account both recall and precision, hence the slight difference. A higher F1 score indicates increased performance of the model.

Finally, in order to determine the true positives and true negatives we create a confusion matrix using the **ConfusionMatixDisplay** function. In the background, the algorithm creates a list of labels with values 0, 1 and 2. Then the **confusion_matrix** is called to produce the matrix with the specified labels. The result can be seen in Figure 6.08 below.

```
labels = [0,1,2]
cm = confusion_matrix(y_test, y_pred, labels=labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot();
```

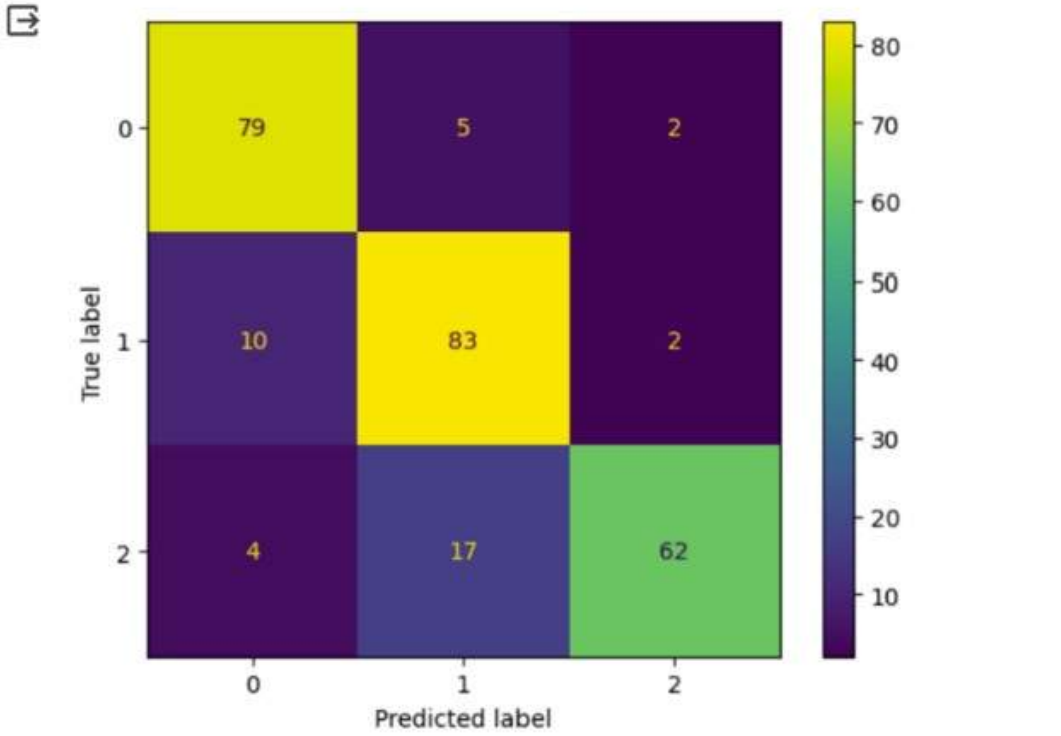


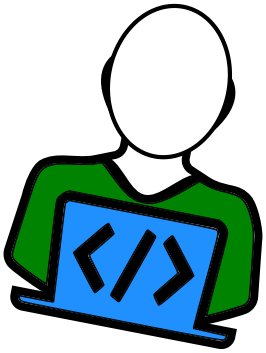
Figure 6.08. Confusion matrix resulting in the true positives and true negatives in Google Colab by Arden University. (Accessible code).

Continue

Topic 2 - Activity 1

Discussion on iLearn

Create a data set for the same scikit-learn data set as demonstrated within the example. Increase the data points to 5000, change the features to 5 and the train / test data ratio to 60% / 40%. Write a mini report on your results and post them to the groups discussion forum for further analysis by your tutors and peers. Do not forget to include any visualisation you have produced.





Import any open-source data set of your choosing and using the example in the lesson, apply appropriate functional changes to result in a full Naïve Bayesian Classification study. Write a mini report on your results and post them to the groups discussion forum for further analysis by your tutor and peers. Do not forget to include any visualisation you have produced.

Continue

Topic X - Activity X

Reading

- Chapter 1: Introduction (pp. 11-31) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido
- Chapter 1: The IDLE Integrated Development Learning Environment (pp. 23-40) from Python for Scientific Computing and Artificial Intelligence by Lynch
- Chapter 1: Preliminaries (pp. 12-30) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney
- Chapter 2: Python Programming Primer (pp. 71-136) from Statistics and Data Visualisation with Python by Rogel-Salazar



Continue

TOPIC 2 SUMMARY

Well done, you have completed Topic 2.

In this particular topic we have given detailed reasoning of how the Naïve Bayesian classifier works and the steps required to build a successful model, based upon the independent contributions of probability. We also defined the three common Classifiers which you will no doubt use within your professional career, namely Bernoulli Naïve, Multi-nominal Bayesian and Gaussian Naïve Bayesian. Having discussed the theory behind the WHY?, we then proceeded to demonstrate a small example based upon small data set.

All theory and small example done; this puts us in an ideal position to take a look at a full Naïve Bayesian example on an imported data set.

See you there.

Continue to Topic 3

Topic 3: FULL example of a Naïve Bayesian classifier using imported data

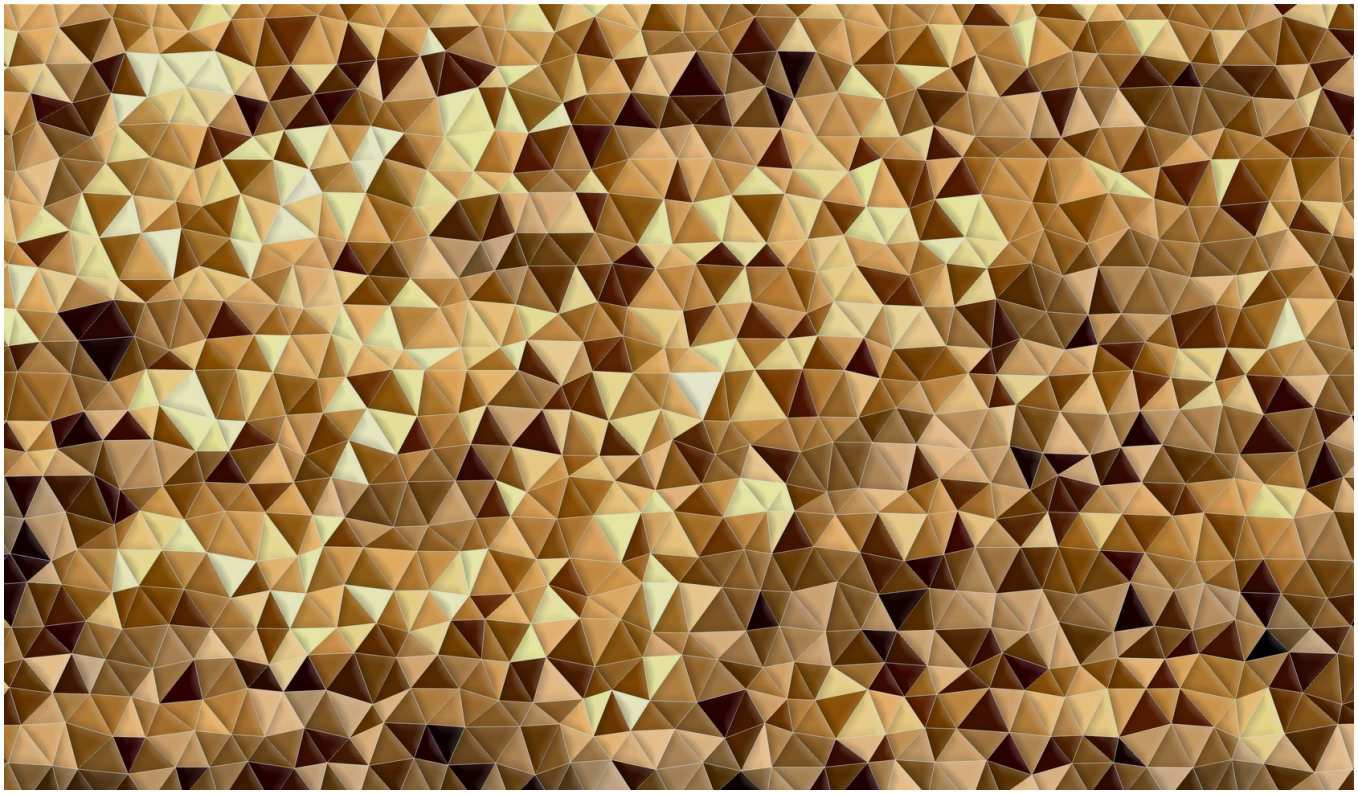


Image from Articulate Rise 360 Content Library.

First of all, we import the relevant libraries, as we have done many times already throughout the module. The seaborn library has been defined as “darkgrid” so that it can help with the visual representations. As shown in Figure 6.09 below.



```
# Importing Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("darkgrid")
```

Figure 6.09. Importing relevant libraries in Google Colab by Arden University. (Accessible code).

Next, we load in our data set so we can use the Naïve Bayesian classifier upon it. The data set which we shall be using for this particular example is the freely available Breast Cancer data set which can be found within Kaggle: Breast Cancer Dataset by M Yasser H.

Here, the data set have been downloaded, converted to a .csv file and imported into Google Colab. The data import is shown below in Figure 6.10, displaying the first five rows.

```
# Printing the first 10 rows of the data set
data = pd.read_csv("/content/breast_cancer.csv")
data.head(5)
```

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	diagnosis
0	17.99	10.38	122.80	1001.0	0.11840	0
1	20.57	17.77	132.90	1326.0	0.08474	0
2	19.69	21.25	130.00	1203.0	0.10960	0
3	11.42	20.38	77.58	386.1	0.14250	0
4	20.29	14.34	135.10	1297.0	0.10030	0

Figure 6.10. Importing data and displaying the first five rows in Google Colab by Arden University.

(Accessible code).

It is important to note the last column on the right within Figure 6.10, i.e., diagnosis. This our target variable and contains the values of 0 and 1. Also, we have five features (columns) and what these five features will do is allow the creation of the model to predict if a patient has breast cancer or not. One final point concerning the data is that all data entries are continuous features and therefore not idea for using the Naïve Bayesian classifier, so we need to modify the data a little: we can either convert the data into categorical features, or fit an approximate distribution, therefore allowing the calculations of the probabilities much easier. For this example, we shall be using the distribution method, but you are encouraged to research the categorical conversion method yourself for a deeper understanding and the differences between both.

With the distribution method, it is advantageous at this point to see how the target variable diagnosis is distributed and to do this, we simply plot a histogram, as shown below in Figure 6.11.

```
# Initial insight 1
data["diagnosis"].hist()
```

```
<Axes: >
```

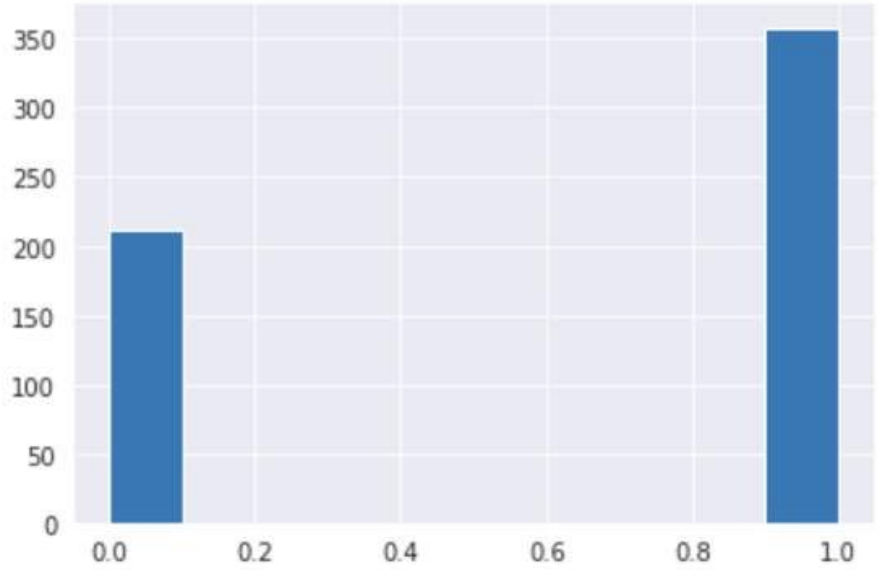


Figure 6.11. Histogram demonstrating the distribution of the target variable in Google Colab by Arden University. (Accessible code).

What Figure 6.11 is showing is that there are more positive examples than negative, meaning that the data is slightly skewed. The next stage is to check and verify that the features are independent: this is very important when trying to employ a Naïve Bayesian classifier model to any given data set.

One way to check for feature independence is to use a heat map, as is shown in Figure 6.12.

```
# Heat Map for feature independence
corr = data.iloc[:,:-1].corr(method="pearson")
cmap = sns.diverging_palette(250,354,80,60,center='dark',as_cmap=True)
sns.heatmap(corr, vmax=1, vmin=-.5, cmap=cmap, square=True, linewidths=.2)
```

```
<Axes: >
```

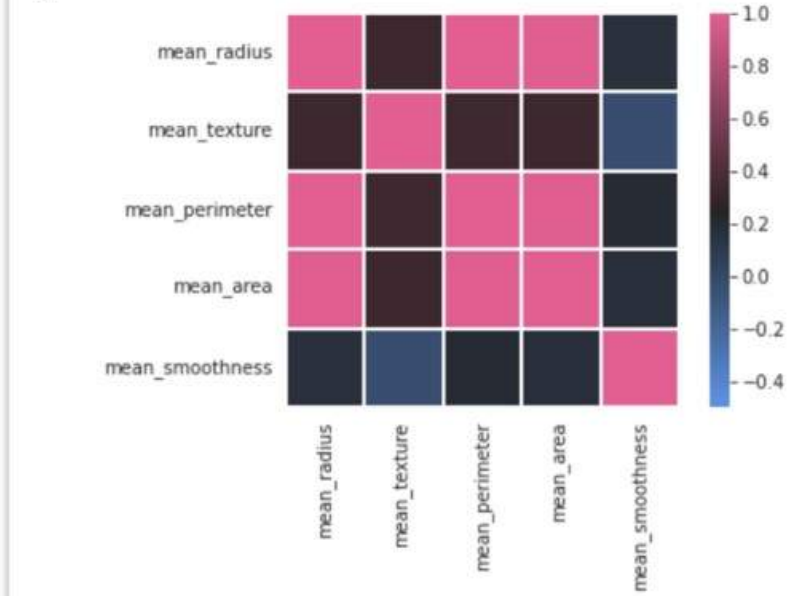


Figure 6.12. Heat map demonstrating feature independence in Google Colab by Arden University. (Accessible code).

What the heat map in Figure 6.12 is informing is that there are many, many positive correlations (pink cells). In fact, apart from the lead diagonal, there are six. However, this is certainly not unusual. The features mean_area and mean_radius are so highly correlated that both including both features would be detrimental to the study, so only one would be included within the model.

Therefore, quickly analysing which features to include within our model, using the same ideology as just stated, we can re-format the data set, as shown below in Figure 6.13.

```
data = data[["mean_radius", "mean_texture", "mean_smoothness", "diagnosis"]]
data.head(10)
```

	mean_radius	mean_texture	mean_smoothness	diagnosis
0	17.99	10.38	0.11840	0
1	20.57	17.77	0.08474	0
2	19.69	21.25	0.10960	0
3	11.42	20.38	0.14250	0
4	20.29	14.34	0.10030	0
5	12.45	15.70	0.12780	0

Figure 6.13. Pre-processing of the data set to include un-correlated features in Google Colab by Arden University. (Accessible code).

At this point it would be advantageous to plot histograms of our features so that we can determine whether the predicted distribution mimics the known distribution of the features, or not. The results can be seen below.

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6), sharey=True)
sns.histplot(data, ax=axes[0], x="mean_radius", kde=True, color='r')
sns.histplot(data, ax=axes[1], x="mean_smoothness", kde=True, color='b')
sns.histplot(data, ax=axes[2], x="mean_texture", kde=True)
```

```
<Axes: xlabel='mean_texture', ylabel='Count'>
```

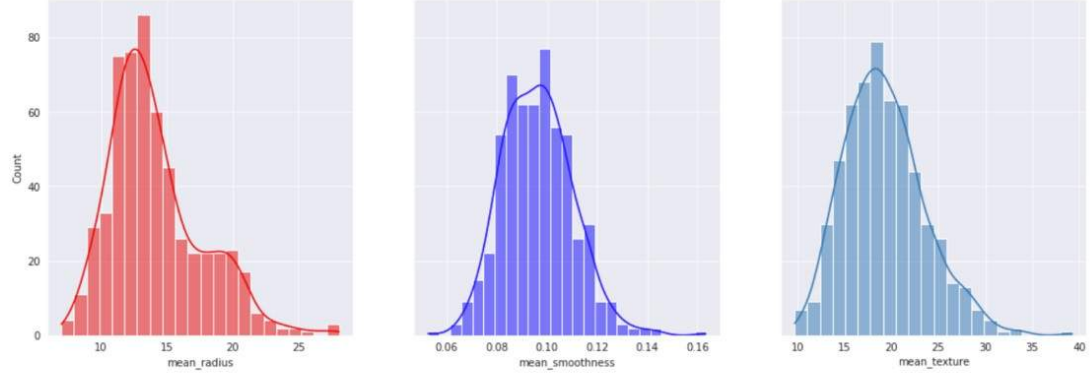


Figure 6.14. Plotting feature distributions in Google Colab by Arden University. (Accessible code).

From Figure 6.14, we can see that the middle and right histograms do follow a normal distribution, therefore can confidently include these features within out model. The left histogram in Figure 6.14 does not really follow a normal distribution that well, but for the purposes of this example we shall use this feature as well, but if you did come across this in a real-world application, you would not include the feature in the model.

Features now determined; we can move onto creating the function that will allow us to determine all of the prior probabilities for all possibilities of *y*. In Python, we can wrap this up into an array that returns all the probabilities. The code for this is shown below.

```
# Defining P(Y=y) for all possible y
def calculate_prior(df, Y):
    classes = sorted(list(df[Y].unique()))
    prior = []
    for i in classes:
        prior.append(len(df[df[Y]==i])/len(df))
    return prior
```

Figure 6.15. Defining the array which will give all possibilities of *y*, using the Naïve Bayesian formula in Google Colab by Arden University. (Accessible code).

Now by using the Gaussian Distribution we can create an additional function that extracts all the data points where the y value is the given label, plus the features names we have targeted. In doing so it will give the probability of *x*, given *y*. The coded function is shown below in Figure 6.16.

```python
# Calculate P(X=x|Y=y) using Gaussian dist.
def calculate_likelihood_gaussian(df, feat_name, feat_val, Y, label):
    feat = list(df.columns)
    df = df[df[Y]==label]
    mean, std = df[feat_name].mean(), df[feat_name].std()
    p_x_given_y = (1 / (np.sqrt(2 * np.pi) * std)) *  np.exp(-((feat_val-mean)**2 / (2 * std**2 )))
    return p_x_given_y
```

Figure 6.16. Creating the function that extracts the data points in y and the associated feature in Google Colab by Arden University. (Accessible code).

Below is the actual Naïve Bayesian classifier working to determine all probabilities, which is why there are loops included.

```python
# P(X=x1|Y=y)P(X=x2|Y=y)...P(X=xn|Y=y) * P(Y=y) for all y and find the maximum
def naive_bayes_gaussian(df, X, Y):
    # get feature names
    features = list(df.columns)[:-1]

    # calculate prior
    prior = calculate_prior(df, Y)

    Y_pred = []
    # loop over every data sample
    for x in X:
        # calculate likelihood
        labels = sorted(list(df[Y].unique()))
        likelihood = [1]*len(labels)
        for j in range(len(labels)):
            for i in range(len(features)):
                likelihood[j] *= calculate_likelihood_gaussian(df, features[i], x[i], Y, labels[j])

        # calculate posterior probability (numerator only)
        post_prob = [1]*len(labels)
        for j in range(len(labels)):
            post_prob[j] = likelihood[j] * prior[j]

        Y_pred.append(np.argmax(post_prob))

    return np.array(Y_pred)
```

Figure 6.17. Naïve Bayesian classifier in Google Colab by Arden University. (Accessible code).

So where are we up to this point? We have imported the libraries we need to complete the model for prediction purposes. Then we identified the features we are to include using both a heat map and a normal distribution analysis. Finally, we have created our Naïve Bayesian classifier. Everything now being checked and verified, we are in a position to actually train and test out model for accuracy. The code and result of the model is shown below in Figure 6.18.

```
# Test Gaussian model
from sklearn.model_selection import train_test_split
train, test = train_test_split(data, test_size=.2, random_state=41)

X_test = test.iloc[:,:-1].values
Y_test = test.iloc[:,-1].values
Y_pred = naive_bayes_gaussian(train, X=X_test, Y="diagnosis")

from sklearn.metrics import confusion_matrix, f1_score
print(confusion_matrix(Y_test, Y_pred))
print(f1_score(Y_test, Y_pred))
```

```
[[36  4]
 [ 0 74]]
0.9736842105263158
```

Figure 6.18. Code and model accuracy in Google Colab by Arden University. (Accessible code).

As can be clearly seen from Figure 6.18, the model has achieved a 97% accuracy, which I am sure you will agree is a pretty good result. In fact, there are only 4 data points where the models' prediction was incorrect and this can be seen from the confusion matrix, again within Figure 6.18.

Continue

Topic 3 - Activity 1

Reading



- Chapter 1: Introduction (pp. 11-31) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido
- Chapter 1: The IDLE Integrated Development Learning Environment (pp. 23-40) from Python for Scientific Computing and Artificial Intelligence by Lynch
- Chapter 2: Python Language Basics, IPython, and Jupyter Notebooks (pp. 30-56) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney
- Chapter 2: Python Programming Primer (pp. 71-136) from Statistics and Data Visualisation with Python by Rogel-Salazar

Continue

TOPIC 3 SUMMARY

Well done, you have completed Topic 3.

Topic 3 has been all about an in-depth Navie Bayesian classifier example. Admittedly, the example is quite involved, however what we ran through will allow you to adopt the approach to all but the most challenging Naïve Bayesian studies in your professional careers. You are encouraged to revisit this particular topic as many times as you need to fully understand each process that was included. Naïve Bayesian classifiers are now almost complete, but there is one more important item to look at, Bayesian Belief Networks and this is what we shall be demonstrating in Topic 4.

See you there.

Continue to Topic 4

Topic 4: Bayesian Belief Network

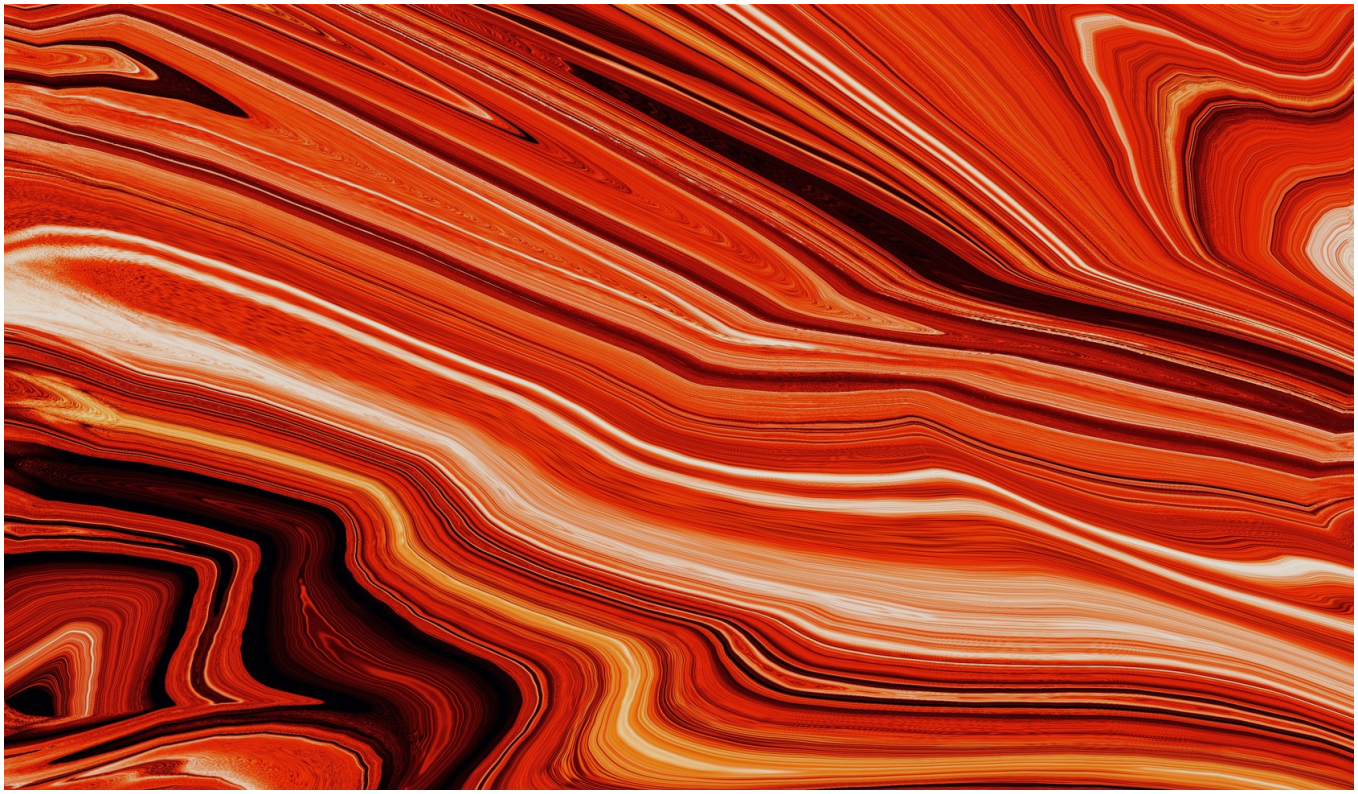


Image from Articulate Rise 360 Content Library.

In keeping with previous topical structures, it is advantageous to give some background context about this methodology, progressing further. Bayesian Belief Networks concerns themselves primarily around probability, especially uncertainty. Therefore, they can be described as a set of variables with conditional dependencies. Bayesian Networks are used within time series predictions for forecasting and anomaly detection within the health industry.

Terminology terms used within Bayesian Networks include Direct Acyclic Graphs (DAG), Conditional Probability Tables (CPT), plus Parental and Child nodes: let us further explain each of these terminologies. DAG is acyclic because the graph does not follow a loop pattern, in other words the conditional probabilities follow a directed path. So, for example, if the weather is overcast outside, you would put on a raincoat and if you put the raincoat on you would also put your wellington boots (wellies) on. This can be graphically represented in Figure 6.19 below.

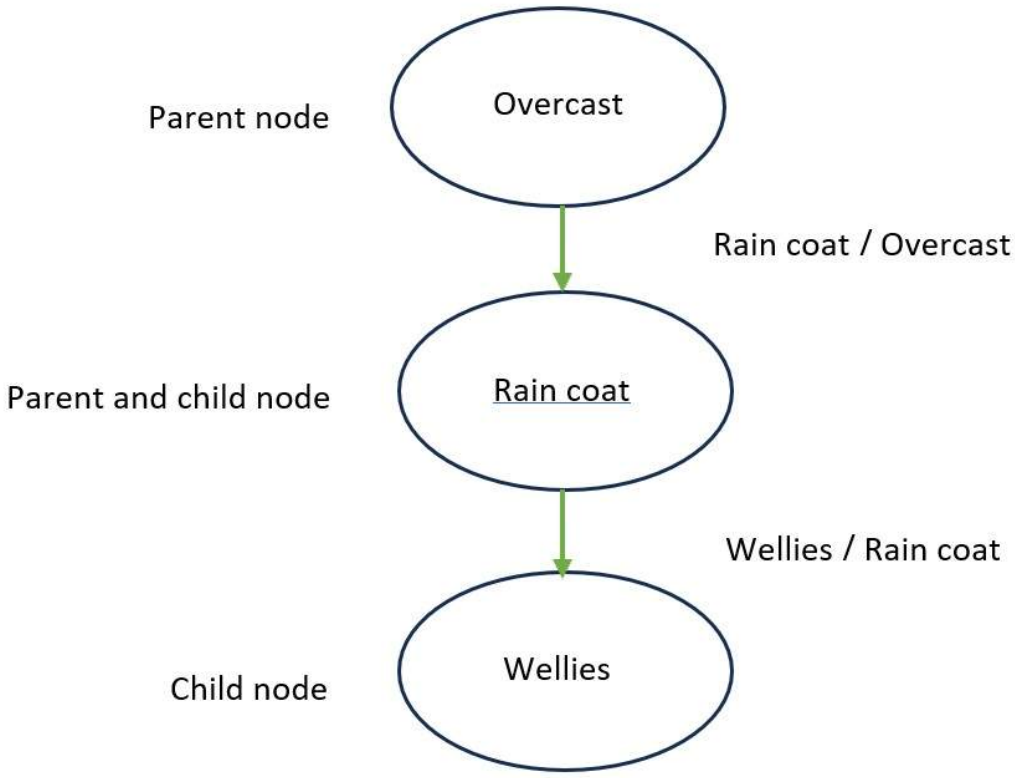


Figure 6.19. Graphical representation of a DAG by Arden University. (Accessible list).

As can be seen from Figure 6.19 above, there is a clear direction to the graph, given by the arrows. There is no closed loop within the graph, which makes it acyclic. Remember though that not on every occasion it is overcast we put on our raincoat and not every time we put on our raincoat, we also put on our wellington boots, these certain conditions are not there, sometimes it might be, but on other times it is not. Therefore, these can be represented as conditional probabilities.

The green arrows within the DAG that gives the directional move of probability from one oval (variable) to another are called nodes, specifically parent and child nodes, which will always have two probabilities. The overcast node is the parent node of the entire DAG. This means that the nodes underneath the overcast node are the child nodes: but the raincoat node is also the parent node to the wellington boots node. It can seem a touch confusing to begin with, but if you follow the DAG in Figure 6.19, it will hopefully make perfect sense. Finally, we have the CPT: this is nothing more than the conditional probabilities for each outcome, which we have to construct and will be demonstrated shortly. The important thing to begin with however is the terminology, as we shall be using this in the remainder of the topic.

Continue

Example

When we show up to the present moment with all of our senses, we invite the world to fill us with joy. The pains of the past are behind us. The future has yet to unfold. But the now is full of beauty simply waiting for our attention.

Suki has moved into a new house and as part of her security policy she has installed a house alarm that activates when there is a suspected burglary, or an earthquake is registered. Suki has asked her new neighbors, Carl and Duban, that if they hear the alarm while she is at work, they contact her immediately, to which they both accept.



Image from Articulate Rise 360 Content Library.

As time passes, it has been noted that Carl always calls Suki when he hears the alarm and sometimes, he mistakes the house phone ringing for the alarm and calls Suki when this happens too. As Duban is a professional musician, he records his music very loud and sometimes misses the alarm. The question is, “What is the probability of the burglar alarm activating, even though no event occurred, given both neighbors called Suki?”

Continue to Solution

Solution

First of all, let's build the DAG so that we can visually represent the problem. This is shown below in Figure 6.20.

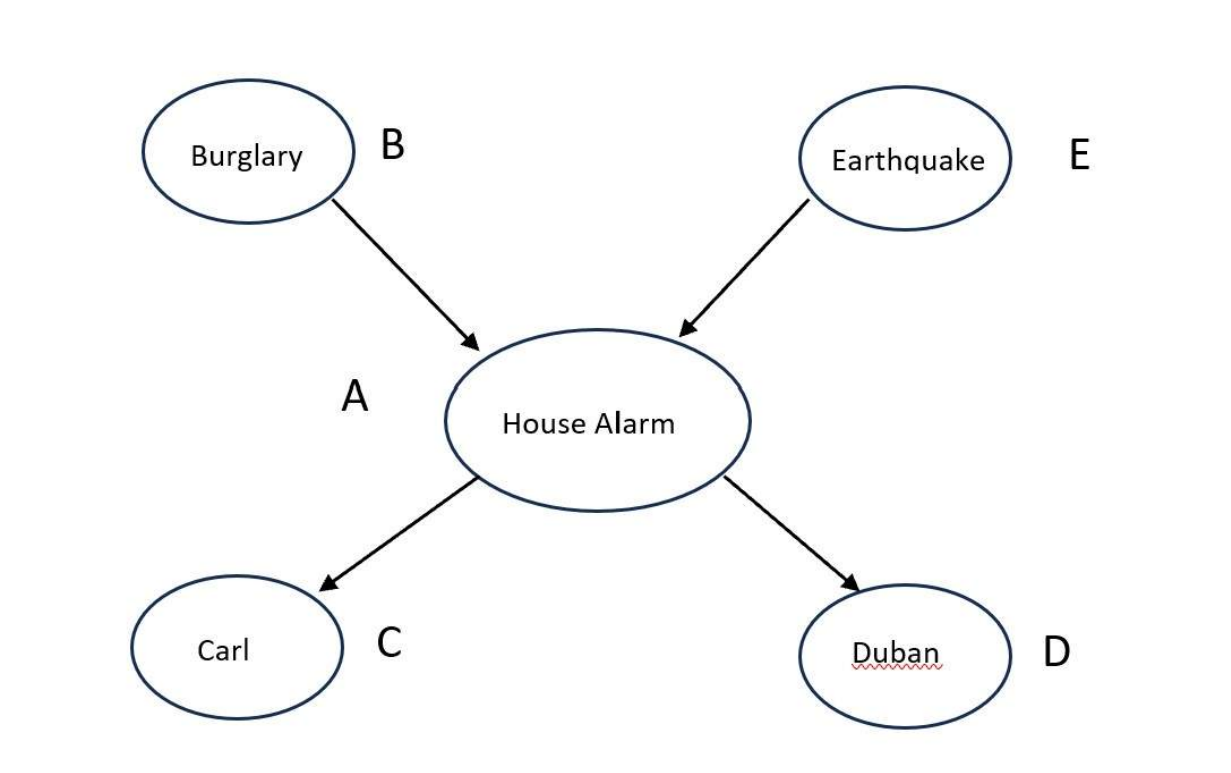


Figure 6.20. Graphical representation of the conditional probability question by Arden University. (Accessible list).

Now we have graphically represented the problem we can begin to build up the individual probabilities, beginning with the overall statement, i.e., P[C, D, A, B, E]. If we take some fictional observed probability for burglary and earthquake, so that we can demonstrate this example fully, we can state that:

$$P(B = T) = 0.02 \textit{ the probability of a burglary}$$

$$P(B = F) = 0.98 \textit{ the probability of not a burglary}$$

$P(E = T)$ = 0.01 *the probability of an earthquake*

$P(E = F)$ = 0.99 *the probability of not an earthquake*

From this we can determine the conditional probability of A (Alarm) which depends upon B & E, which can be summarized in a table, which is shown below.

Conditional probability for Alarm

B	E	P(A = True)	P(A = False)
True	True	0.94	0.06
True	False	0.95	0.05
False	False	0.31	0.69
False	False	0.001	0.999

Next, we can determine the conditional probability that Carl will call dependent on the alarm.

Conditional Probability for Carl calls

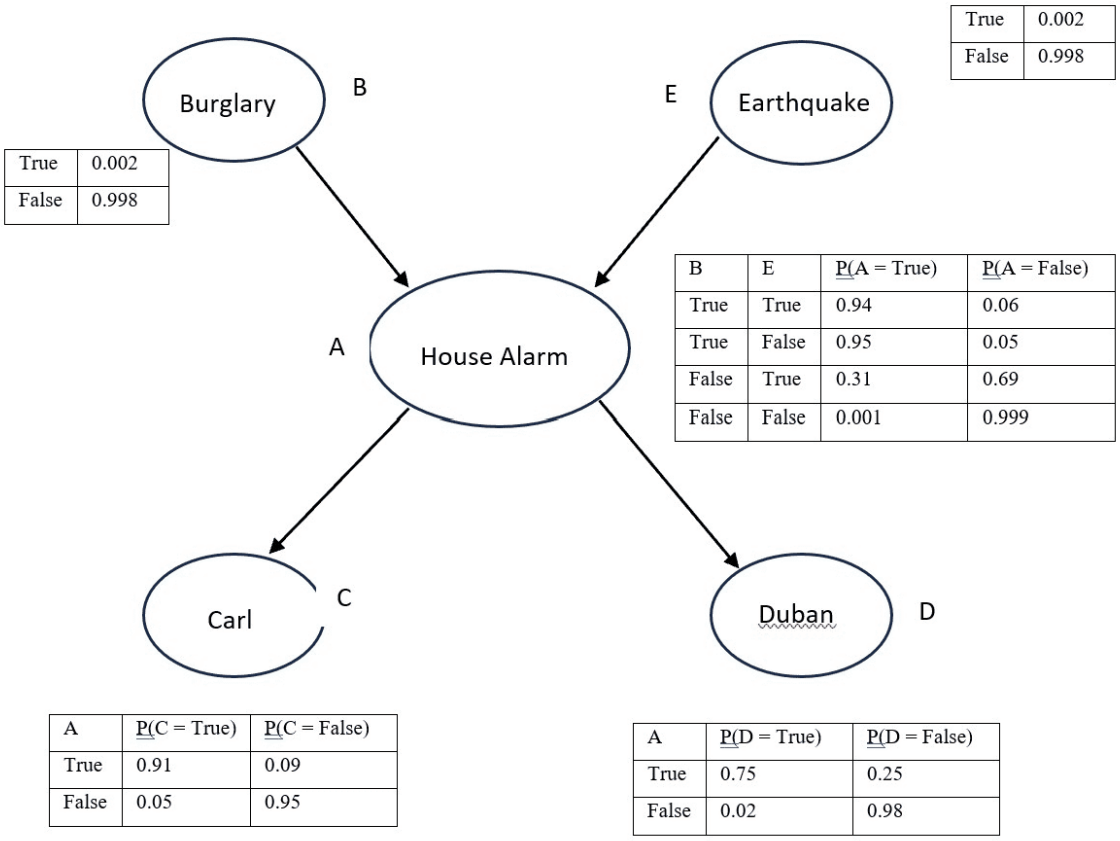
A	P(C = True)	P(C = False)
True	0.91	0.09
False	0.05	0.95

The same conditional probability can also be determined for Duban, remembering that this will also depend on the parent node, Alarm.

Conditional Probability for Duban calls

A	P(D = True)	P(D = False)
True	0.75	0.25
False	0.02	0.98

To make this a touch clearer, before we go to the solution, we can put the conditional probabilities next to the associated nodes, as shown below in Figure 6.21.



True	0.002
False	0.998

True	0.002
False	0.998

B	E	P(A = True)	P(A = False)
True	True	0.94	0.06
True	False	0.95	0.05
False	True	0.31	0.69
False	False	0.001	0.999

A	P(C = True)	P(C = False)
True	0.91	0.09
False	0.05	0.95

A	P(D = True)	P(D = False)
True	0.75	0.25
False	0.02	0.98

Figure 6.21. Full DAG with associated conditional probabilities by Arden University.

Having determined all the conditional probabilities, we can find out the probability to our original question, as a reminder:

> ***"What is the probability of the burglar alarm activating, even though no events occurred, given both neighbors called Suki?"***

First of all, lets define the probability formally, shown below:

$$p(C, D, A, \neg B, \neg E) = P(C|A) \cdot P(D|A) \cdot P(A|\neg B \wedge \neg E) \cdot P(\neg B) \cdot P(\neg E)$$

$$= 0.75 \cdot 0.91 \cdot 0.001 \cdot 0.998 \cdot 0.999 = 0.00068045 \textit{ or } 0.06\%$$

Now that we have run through an example graphically. Below demonstrates a coded example using a small imported data set. You will be familiar with many of the commands and function included, therefore work your way through the example at your own pace.

Continue

Example

```
import numpy as np
import pandas as pd
from pgmpy.models import BayesianModel
from pgmpy.estimators import MaximumLikelihoodEstimator, BayesianEstimator
from pgmpy.inference import VariableElimination
from pgmpy.factors.discrete import TabularCPD
```

```
[8] data = pd.read_csv('/content/Lesson_6_Topic_4.csv')
```

```
[10] data = data.dropna()
```

```
[11] data.head()
```

	Age	Gender	Purchase	History
0	32	m	1200	h
1	25	f	800	m
2	41	m	1500	l
3	35	f	1000	h
4	28	m	900	m

Figure 6.22a. Coded example in Google Colab by Arden University. (Accessible code).

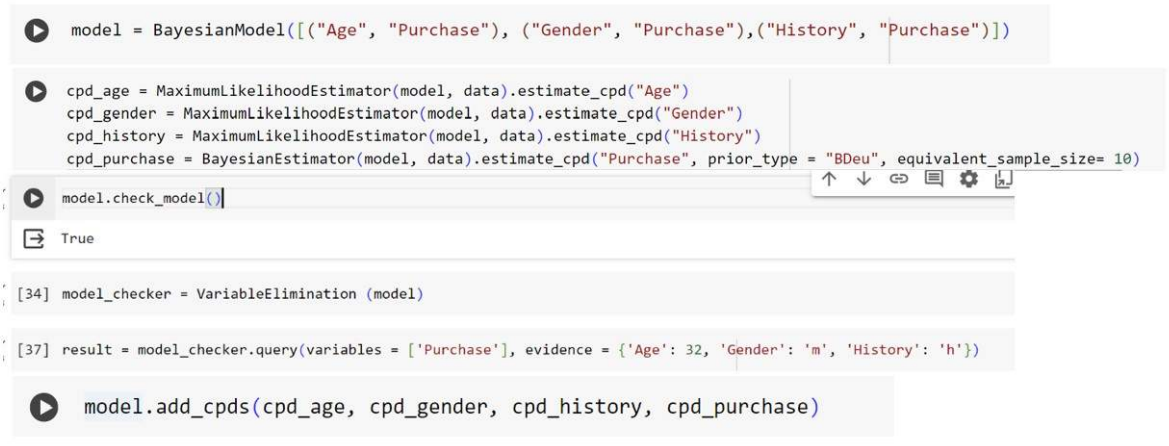


Figure 6.22b. Coded example in Google Colab by Arden University. (Accessible code).

```
print(result)
```

```
+-----------------+------------------+
| Purchase        |   phi(Purchase) |
+=================+==================+
| Purchase(500)   |           0.0000 |
+-----------------+------------------+
| Purchase(800)   |           0.0000 |
+-----------------+------------------+
| Purchase(900)   |           0.0000 |
+-----------------+------------------+
| Purchase(1000)  |           0.0000 |
+-----------------+------------------+
| Purchase(1200)  |           1.0000 |
+-----------------+------------------+
| Purchase(1300)  |           0.0000 |
+-----------------+------------------+
| Purchase(1500)  |           0.0000 |
+-----------------+------------------+
| Purchase(3940)  |           0.0000 |
+-----------------+------------------+
| Purchase(4440)  |           0.0000 |
+-----------------+------------------+
```

Figure 6.22c. Coded example in Google Colab by Arden University. (Accessible code).

This example has demonstrated how to build and assess a Bayesian Belief classifier. The result above shown that the model has a 100% accuracy, based upon the query we stated within the code.

Hurrah!

Continue

Topic 4 - Activity 1

Reading



- Chapter 2: Supervised Learning (pp. 33-102) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido
- Chapter 3: Jupyter Notebooks and Google Colab (pp. 53-64) from Python for Scientific Computing and Artificial Intelligence by Lynch

- Chapter 2: Python Language Basics, IPython, and Jupyter Notebooks (pp. 30-56) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney
- Chapter 3: Snakes, Bears & Other Numerical Beasts: NumPy, SciPy & pandas (pp. 137-178) from Statistics and Data Visualisation with Python by Rogel-Salazar

Continue

TOPIC 4 SUMMARY

Well done, you have completed Topic 4.

Within this final topic we demonstrated what a Bayesian Belief Network is. As we saw, the network itself follows a Direct Acyclic Graph, for reasons discussed within the topic: in addition to the correct construction of associated Conditional Probability Tables. All of which were detailed graphically within the topic. Once the topic had been discussed theoretically, we hopefully solidified knowledge through a coded example

All topics have now been concluded within this lesson.

Continue to Knowledge check

Knowledge check

Select the correct answers to the following questions.

1 of 2: What are Naïve Bayesian classifiers?

- ◯ A classification algorithm based on Bayes' Theorem
- ◯ A collection of classification algorithms based on Bayes' Theorem
- ◯ A classification algorithm based on Wards Theorem

SUBMIT

2 of 2: What are Multinomial Naïve Bayesian Classifiers predominately used for?

- Numerical anomaly detection
- Regression anomaly detection
- Text anomaly detection

SUBMIT

Continue to Lesson summary

Lesson summary

LESSON SUMMARY

The initial topic on Naïve Bayesian was concerned with not only introducing the topic, but also its graphical and mathematical representation. Hopefully, throughout the topic you fully understood the basic principles behind the classifier which allowed us to develop a little further and how you use the Naïve Bayesian classifier effectively. Topic 2 gave a detailed reasoning of how the Naïve Bayesian classifier works and the steps required to build a successful model, based upon independent contributions of probability. We defined three common Classifiers, namely Bernoulli Naïve, Multi-nominal Bayesian and Gaussian Naïve Bayesian. Finally, we demonstrated, via a small example how to apply the Gaussian classifier

Topic 3 was all about an in-depth Navie Bayesian classifier example. Challenging, yes, however what we ran through will allow you to adopt the approach to all but the most challenging Naïve Bayesian studies in your professional careers. Within the final topic we demonstrated what a Bayesian Belief Network is. As we saw, the network itself follows a Direct Acyclic Graph, for reasons discussed within the topic: in addition to the correct construction of associated Conditional Probability Tables. All of which were detailed graphically within the topic.

All topics have now been concluded within this lesson. Hopefully you have found the lesson challenging and learned many new things, especially around Naïve Bayesian Classifiers.

See you in the next lesson.

Continue to 'Reading list'

Reading List

All reading and resources in this module are provided by the Arden University Library service. The full reading list for this module can be accessed at the link below. More books, journals and databases can be found on the Library Portal. If you need these resources in a different format, our accessibility tools can help.

Full reading list

You can find all the key, essential and wider reading and resources for this module on the dedicated reading list page on the Learning Success Hub.

VIEW READING LIST

References

Cited sources that do not appear in your reading list:

PR Newswire, (2016). Netflix Creates $1 Million Netflix Prize to Promote Progress in Recommendation Systems. *PR Newswire* [online], 2 October 2006.

University of Cambridge, (2013). Digital records could expose intimate details and personality traits of millions. *University of Cambridge* [online], 11 March 2013. Cambridge: University of Cambridge.

Valdiviejas, H. and Bosch, N., (2020). Using Association Rule Mining to Uncover Rarely Occurring Relationships in Two University Online STEM Courses: A Comparative Analysis [online]. *In:* Rafferty, A. N., Whitehill, J., Romero, C. and Cavalli-Sforza, V., eds. *13th International Conference on Educational Data Mining* [online], 10-13 July 2020. Massachusetts, USA: International Educational Data Mining Society.